



A FORCE OS

Product & Technical Specification

A complete specification of AForce OS — a real-time human-performance operating system that turns hydration into measurable readiness, recovery, and a daily ritual.

PAUSE → HYDRATE → LOCK IN → PERFORM

VERSION

1.0.0

PLATFORM

iOS & Android

BUNDLE ID

com.aforce.os

COMPILED

June 19, 2026

CONFIDENTIAL — FOR DISCUSSION & PLANNING PURPOSES

CONTENTS

Specification Index

PART I

Product & Technical Specification

- At a Glance
- 1. Platform & Technology Stack
- 2. Application Architecture
- 3. Navigation & Screen Inventory
- 4. Onboarding & Cold-Launch Opening Sequence
- 5. Hydration Scoring Engine
- 6. Recovery & Health Signals
- 7. AI Coach / Voice Engine
- 8. Social Layer
- 9. Store & Subscription System
- 10. Authentication & Identity
- 11. Design System
- 12. Localization
- 13. Feature Flags
- 14. Backend & API
- 15. Build, Distribution & Permissions
- Appendix — Key Dependency Versions

PART II

Social Mode — Safety & Estimation Spec

- 1. Voice & tone
- 2. BAC estimation (Widmark approximation)
- 3. Impairment escalation matrix
- 4. Disclaimer policy
- 5. Recovery Mode
- 6. Orb overlay
- 7. Test invariants

Product & Technical Specification

A complete specification of the **AForce OS** mobile application — a real-time human-performance operating system delivered as a React Native / Expo app with an Express 5 + PostgreSQL backend. Its purpose: turn hydration into measurable readiness, recovery, and a daily ritual — *Pause → Hydrate → Lock In → Perform*.

At a Glance

App name	AForce OS
Version	1.0.0 (iOS build 1 / Android versionCode 1)
Platform	iOS + Android (React Native / Expo), portrait, dark UI
Bundle ID	<code>com.aforce.os</code> (iOS & Android)
Frontend	Expo SDK ~54, React Native 0.81.5, Expo Router 6
Backend	Node.js, Express 5, PostgreSQL, Drizzle ORM, Zod
Auth	Clerk (<code>@clerk/expo</code>)
Payments	Stripe + stripe-replit-sync
Languages	6 live (EN/ES/FR/DE/PT/IT) + 5 wired-but-hidden
Visible tabs	Home · Hydration · Protocols · Community · Profile

1. Platform & Technology Stack

- **Framework:** Expo SDK `~54.0.27` , React Native `0.81.5` (New Architecture enabled, React Compiler enabled, typed routes).
- **Navigation:** Expo Router `~6.0.17` (file-based; both classic `Tabs` and native `NativeTabs`).
- **State management:** Zustand store organized into reducer-style slices (`store/useAppStore.tsx` , `store/appStoreReducer.ts` , `store/slices.tsx`), plus a dedicated cart store (`store/useCartStore.tsx`).
- **Data fetching:** `@tanstack/react-query` , consuming a generated OpenAPI client (`@workspace/api-client-react`).
- **Animation:** React Native Reanimated `~4.1.1` + Worklets.
- **Internationalization:** `i18next` + `react-i18next` + `expo-localization` .
- **Hardware / device:** `expo-camera` (HydroScan), `expo-location` (weather), `@kingstinct/react-native-healthkit` (Apple Health), `expo-haptics` , `expo-audio` , `expo-notifications` , `expo-secure-store` .
- **Typography assets:** Inter, Bebas Neue, DM Sans (`@expo-google-fonts/*`).

2. Application Architecture

- `app/` — root layouts, the authenticated tab group, gated routes, modals.
- `components/` — reusable UI (orb, rails, opening sequence, etc.).
- `services/` — business logic (scoring, voice, health, social, i18n).

- `store/` — slice-based Zustand state + reducer.
- `utils/` — pure, dependency-free helpers (scoring math, units, dashboards).
- `featureFlags/` — feature toggles controlling surface exposure.
- `theme/` — WHOOP-Cinematic color system, typography, spacing, radii.
- `data/` — products, subscription plans, templates, mock data.
- `types/` — shared TypeScript definitions.
- `locales/` — translation resources.

Governance principle: *Build 100% · Show 10% · Unlock over time.* The engine expands; navigation stays fixed. Feature flags gate exposure.

3. Navigation & Screen Inventory

Visible tab bar (5 tabs)

1. **Home** — readiness + hydration command dashboard (`app/(tabs)/index.tsx`).
2. **Hydration** — chronological hydration/recovery feed (`app/(tabs)/journal.tsx`).
3. **Protocols** — AForce Protocol guidance (`app/(tabs)/protocol.tsx`).
4. **Community** — rankings, challenges, battles, teams, map (`app/(tabs)/competition.tsx`).
5. **Profile** — profile + settings (`app/(tabs)/profile.tsx`).

Hidden routes (`href: null` — built, not surfaced)

`scan` (HydroScan), `social` (Social Mode), `sleep` (Sleep Mode), `social-legacy` (developer-only).

Stack / modal routes

`/store`, `/cart`, `/subscription`, `/achievements`, `/onboarding`, `/circles`, `/territory`, `/ring`, `/heat`.

4. Onboarding & Cold-Launch Opening Sequence

- **Opening sequence** (`components/opening/OpeningSequence.tsx`) — a 4-stage cinematic that plays **once per cold launch** as an overlay (touches no routing): 1. White water-drop symbol, breathing fade. 2. AForce wordmark + brand-red hairline + "Performance Is Non-Negotiable". 3. PAUSE / HYDRATE / LOCK IN / PERFORM ritual stagger. 4. "TODAY'S READINESS" + count-up to the live score, with a band-aware caption (a depleted user reads "REHYDRATE NOW", never "READY TO PERFORM"). Slow Apple-Vision-Pro pacing, tap-to-skip, fully reduced-motion aware. Display-only (never mutates score).
- **First-run onboarding** (`app/onboarding.tsx`) — Goal selection (Performance / Recovery / Endurance / Balance / Longevity) → Activity level (Sedentary → Athlete) → Profile setup (weight / height / sex). Gated by two flags (`hasSeenWelcome` + `hasCompletedOnboarding`).

5. Hydration Scoring Engine

Files: `services/hydrationScoreService.ts`, `utils/hydrationScore.ts`, `utils/scoringEngine.ts`.

- **Base unit:** 1 unit = 12 oz.
- **Point values:**
- Water: +0.5 pts/oz (+6 per unit).
- AForce Berry Blast / Watermelon Surge: +10 pts.

- AForce Soursop Edge: +11 pts.
- **Contextual bonuses:**
- *Heat Guard*: +2 for Watermelon Surge when heat score ≥ 45 .
- *Depleted boost*: +2 for Soursop Edge when score < 40 .
- *Hydration Cycle*: +8 to +20 for logging water + AForce within 30 min (larger bonus at lower scores).
- **Absorption caps**: max 1.5 units per rolling 20-min window; excess absorbed at 75% efficiency.
- **Release curves:**
- Water: 60% immediate, 40% released over ~12.5 min.
- AForce: 70% immediate, 30% released over ~25 min.
- **Performance states**: PEAK (90–100), BALANCED (75–89), RECOVERING (60–74), DEPLETED (0–59).
- **Score Protection rule**: only *completed* behavior changes the score. Recommendations, scans (HydroScan is advisory), and product selection never increase the score.

6. Recovery & Health Signals

Files: `services/healthConnection.ts`, `services/recoveryEngine.ts`, `utils/biometricsAggregator.ts`.

- **Providers**: Apple Health (HealthKit), Samsung Health (SDK), WHOOP (OAuth), merged "freshest-wins".
- **Recovery Capacity engine**: derives a 0–100 score from hydration score minus penalties (sleep loss, dark urine, drink load) plus a streak boost.
- **Tracked signals**: thirst (1–5), energy (1–5), urine color (1–8), steps, temperature, humidity, workout load.
- **Apple Health entitlements**: reads heart rate, HRV, sleep, and workouts; writes hydration logs back to Health.

7. AI Coach / Voice Engine

Files: `services/elevenLabsTts.ts`, `services/voiceService.ts`, `services/intentClassifier.ts`.

- **Stack**: ElevenLabs TTS proxied through the API server (`/api/voice/tts`), with `expo-speech` as fallback.
- **Lifecycle**: classify transcript intent (e.g., `LOG_INTAKE`, `COMPLETE_CYCLE`) → resolve persona by performance band → render template → TTS output.
- **Persona modes**: tone and phrasing shift with hydration band (more decisive in lower bands); verdict-aware comparisons after HydroScans.
- **Water-First copy lock**: coach recommendations always begin with `HYDRATE NOW` / `Start with water`; optional product support is only suggested after hydration needs are evaluated.

8. Social Layer

Files: `services/circleService.ts`, `services/recoveryCircle.ts`, `services/territoryEngine.ts`.

- **Circles**: accountability groups for friends and teams.
- **Recovery Circle**: private cohort (max 3 people) with fixed checkpoints (Day 0, 1, 3, 7, 30).
- **Territory**: competitive regional scoring — $\text{Avg Performance} \times 0.35 + \text{Protocol Rate} \times 0.25 + \text{Streak Density} \times 0.15 + \text{Recovery Efficiency} \times 0.15 + \text{Momentum} \times 0.10$.
- **Social Mode**: alcohol intake affects hydration score and decay rate.

9. Store & Subscription System

Files: `data/subscriptionPlans.ts` , `types/subscription.ts` . Plans inherit features via an inheritance chain; feature flags map to `featureFlags/flags.ts` .

Consumer

PLAN	PRICE	NOTES
Core	\$9.99/mo	Entry: OS access, AI commands, daily tracking.
Recovery+	\$9.99/mo	Standalone add-on — unlocks Recovery Mode after Social Mode.
AForce Athlete	\$19.99/mo	Personalized protocols, advanced recovery, competition access.
Performance Bundle	\$59.99/mo	Best value — Athlete + monthly product drop (1 canister or 2 stick packs).
AForce Elite	\$99/mo	Guardian Mode for individuals + full monthly bundle (canister + sticks + RTDs) + concierge.

Team / Program

PLAN	PRICE	SEATS
Team Core Starter	\$49/mo	up to 25
Team Core Growth	\$99/mo	up to 50
Team Core Pro	\$149/mo	up to 100

Performance Systems — Clutch Access

PLAN	PRICE
Clutch Starter	\$1,000/mo
Clutch Pro	\$2,500/mo
Clutch Elite	\$5,000/mo

Performance Systems — Guardian

PLAN	PRICE	TERMS
Guardian Core	\$5,000/mo	+ \$7,500 setup, 6-month minimum
Guardian Elite	\$8,000/mo	+ \$12,500 setup, 12-month minimum

- **Pricing security:** pricing, shipping, and tax are computed server-side; Stripe is the source of truth, mirrored to PostgreSQL via webhooks. Webhook signatures are verified.
- **Entitlements:** `useEntitlement.ts` gates paywalled surfaces by plan tier.

10. Authentication & Identity

- **Provider:** Clerk — `@clerk/expo` (mobile), `@clerk/express` (server).
- **Sign-in:** custom email/password + Google SSO.
- **Gating:** the `(tabs)` group is gated behind `isSignedIn` ; `ClerkProvider` mounts at the root layout. Auth-gated routes exist on both client and server.

- **Token bridging:** Clerk session tokens bridge into the OpenAPI client.
-

11. Design System

Files: `theme/colors.ts`, `theme/typography.ts`, `theme/spacing.ts`.

- **Palette (WHOOOP-Cinematic):** pure black `#000000` canvas, AForce brand red `#FF3B30` (sparse accents), WHOOOP lime / Peak Green `#B6FF00`, plus WHOOOP recovery colors (green / yellow / red).
 - **Typography:** Inter (400 → 800), tracked labels (letter-spacing up to "ultra").
 - **Spacing:** 4px grid (scale 1–24; e.g. step 8 = 32px).
 - **Aesthetic:** content floats on pure black, big numbers / small tracked labels, soft radial glows (no hard box shadows), generous spacing.
 - Tokens exported in Tokens Studio format (`design/aforce-tokens.json`) and a human-readable spec (`design/aforce-design-tokens.md`).
-

12. Localization

- **Live (visible in selector):** English, Spanish, French, German, Portuguese, Italian.
 - **Wired but hidden behind flags:** Arabic, Chinese, Japanese, Korean, Hindi.
 - **Lock:** no country-specific prioritization; architecture stays modular so new languages add without a rebuild. Files: `services/i18nService.ts`, `locales/`.
-

13. Feature Flags

File: `featureFlags/flags.ts`. Two profiles: `DEFAULT_FLAGS` and `DEMO_ALL_ON_FLAGS`. Representative flags: `clutch_access_enabled`, `guardian_intelligence_enabled`, `phantom_wearable_enabled`, `cruise_mode_enabled`, `global_leaderboard_enabled`, `team_roster_enabled`. Flags map to subscription feature gating so entitlements stay consistent.

14. Backend & API

- **Server:** Express 5 with Zod input/output validation derived from an OpenAPI spec (Orval generates the React Query client + Zod schemas).
 - **Database:** PostgreSQL via Drizzle ORM.
 - **Concurrency:** `SELECT ... FOR UPDATE` to serialize concurrent user actions.
 - **Real-time:** REST mutations broadcast over a shared HTTP/WebSocket server.
 - **Integrations:** Stripe (+ stripe-replit-sync), OpenWeather (proxied with an in-memory TTL cache + rate limiting), ElevenLabs voice (`/api/voice/tts`).
 - **Logging:** structured logging for request and non-request code.
 - **Scaling:** designed for horizontal scaling.
-

15. Build, Distribution & Permissions

- **Build service:** EAS Build — `development`, `preview`, `production` profiles (`eas.json`).

- **Bundle IDs:** iOS & Android `com.aforce.os` ; iOS `supportsTablet: false` .
- **Versioning:** app version 1.0.0; iOS buildNumber 1; Android versionCode 1.
- **App config:** dark UI, portrait, black splash, notification accent `#B6FF00` .
- **Permissions:**
 - iOS — Camera (barcode scan), Location When In Use (weather), HealthKit (read HR/HRV/sleep/workouts; write hydration logs).
 - Android — `CAMERA` , `ACCESS_COARSE_LOCATION` , `ACCESS_FINE_LOCATION` .
- **Build commands** (from `artifacts/aforce-os/`): `eas:build:ios` , `eas:build:android` , `eas:build:all` , `eas:build:preview` , `eas:build:dev` ; submit via `eas:submit:ios` / `eas:submit:android` .

Appendix — Key Dependency Versions

PACKAGE	VERSION
expo	~54.0.27
react-native	0.81.5
expo-router	~6.0.17
react-native-reanimated	~4.1.1
@clerk/expo	3.2.2
@kingstinct/react-native-healthkit	^14.0.0
@tanstack/react-query	catalog
i18next	^26.0.6
react-i18next	^17.0.4
expo-camera	~17.0.10
expo-audio	~1.1.1
expo-speech	~14.0.8
lucide-react-native	^1.16.0
typescript	~5.9.2

Specification compiled from the AForce OS codebase. Internal — for discussion and planning purposes.

Social Mode — Safety & Estimation Spec

This document is the canonical reference for the BAC estimator, impairment escalation, transportation prompts, and recovery flow that power **AForce OS Social Mode**. Anyone touching the engine, UI, or copy must read this first.

1. Voice & tone

Social Mode is **calm, protective, never preachy**.

- The system **never** moralizes, lectures, or shames the user for drinking. It does not use words like "stop drinking", "you should not", or "irresponsible".
- The system **never** says the user is "safe to drive" or implies legal compliance. The strongest positive language allowed is "your estimate is currently low" — it never crosses into permission.
- When escalating, the system uses protective verbs: *plan a ride, switch to water, arrange safe transport, stop alcohol intake*. It describes outcomes, not character.
- Headlines are short. Bodies are 1–2 sentences max.

2. BAC estimation (Widmark approximation)

Implemented in `services/bacEstimationService.ts`.

```
gramsAlcohol = sum( drink.oz * (drink.abv/100) * 0.789 * 29.5735 )
bodyMassG    = bodyWeightLbs * 453.592
r             = 0.68 (male / unspecified) | 0.55 (female)
foodFactor   = 0.92 if ateRecently else 1
bacRaw       = (gramsAlcohol / (bodyMassG * r)) * 100 * foodFactor
elapsedH     = hours since first drink
bacCurrent   = max(0, bacRaw - 0.015 * elapsedH)
```

The output is a **range** widened by ± 0.01 around the point estimate. We never display a single false-precision number.

FIELD	HOW IT'S DERIVED
<code>rangeLow</code> / <code>rangeHigh</code>	Point estimate ± 0.01 .
<code>trend</code>	Compare current BAC to BAC 15 min ago; $ \Delta < 0.005$ = <code>steady</code> .
<code>confidence</code>	<code>high</code> if sex is provided AND $\geq 50\%$ of drinks have explicit oz/abv AND ≤ 8 drinks.
<code>timeToClearMinutes</code>	$(\text{bacCurrent} - 0.005) / 0.015 * 60$, rounded up to nearest 5 min.

Inputs

- `drinks`: { `type`, `loggedAt`, `abv?`, `oz?` }[] — `abv` / `oz` fall back to the catalog defaults when omitted.
- `bodyWeightLbs`: floored to 80 lbs to avoid divide-by-tiny errors.
- `sex`: optional. `'male'` | `'female'` | `'unspecified'`.
- `ateRecently`: optional boolean.

Drink catalog (`data/alcoholDrinks.ts`)

TYPE	DEFAULT OZ	DEFAULT ABV	DECAY MULTIPLIER	SUGAR LOAD
beer	12	5.0	1.15	3
wine	5	12.5	1.20	4
cocktail	8	14.0	1.30	8
liquor	1.5	40.0	1.35	1
hard_seltzer	12	5.0	1.15	1
custom	6	12.0	1.25	4

3. Impairment escalation matrix

Implemented in `services/legalSafetyService.ts`. Mapping uses the **midpoint** of the BAC range so trend is smooth across refreshes.

BAC MIDPOINT	LEVEL	SAFETY CARD	STOP DRINKING?	COACH COMMAND
< 0.030	LOW	hidden	no	(standard hydration / pacing copy)
0.030 – 0.049	ELEVATED	hidden	no	(standard hydration / pacing copy)
0.050 – 0.079	MODERATE	shown — caution	no	"Plan a ride before your next drink"
0.080 – 0.119	HIGH	shown — warning	yes	"Do not drive. Use a rideshare."
≥ 0.120	CRITICAL	shown — critical	yes	"Stop alcohol intake. Recovery req."

The safety card is hidden at **LOW** and **ELEVATED** so the user only sees the legal-protection language when it actually applies. The "stop drinking" sub-prompt only appears at HIGH/CRITICAL.

4. Disclaimer policy

Every surface that shows a BAC value, impairment level, transportation prompt, or recovery time **must** render the standard disclaimer pair:

Estimate only · Not a legal or medical determination.

The i18n keys are `social.estimate_only` and `social.not_legal_medical`. They are translated in all 6 supported locales (`en`, `es`, `fr`, `de`, `pt`, `it`).

The system **never**:

- Tells the user they are "safe to drive".
- Promises a specific BAC at a specific future time.
- Asserts compliance with any legal limit (those vary by jurisdiction and by individual physiology).
- Provides medical advice (kidney/liver concerns, medication interactions, etc.).

5. Recovery Mode

Triggered when the user taps **End Night** in the Social Mode sheet. Engine sets `socialMode.endedAt` and the rollup flips `inRecoveryWindow = true` for the next 8 hours (`RECOVERY_WINDOW_MS = 8 * 60 * 60 * 1000` in `socialModeEngine.ts`).

While in recovery the UI renders `RecoveryModeCard` showing:

1. The estimated time until BAC clears (from `bac.timeToClearMinutes`).
2. A 3-step morning protocol — water → AForce RTD → 7+ hours sleep.
3. The standard disclaimer pair.

The home banner shifts to amber and reads `social.recovery_active` .

6. Orb overlay

`StatusPulse0rb` accepts `socialOverlay?: { alcoholLoad: number; unstable: boolean }`.

- `alcoholLoad` $\in [0, 1]$ is derived from the active decay multiplier and pushes a subtle violet outer ring.
- `unstable = true` when impairment is HIGH or CRITICAL — the ring flips crimson and pulses faster.

The overlay is purely additive; it never replaces the normal hydration gradient.

7. Test invariants

`services/__tests__/bacEstimation.test.ts` pins:

- Zero drinks → zero BAC, zero clear time.
- One beer → LOW band.
- Four quick liquor shots → at least MODERATE.
- Trend: rising right after drinks, falling after long elimination.
- Time-to-clear is non-negative and a multiple of 5 minutes.
- Confidence degrades when sex is unspecified.
- Food intake softens the BAC estimate vs an empty stomach.
- Safety prompt is hidden at LOW/ELEVATED.
- Safety prompt escalates to "do not drive" at HIGH and CRITICAL.
- Safety disclaimer key is always returned.

These invariants must continue to hold after any change to the engine or the impairment thresholds.